

Egocentric Capture QA

Technical Documentation

Quality assurance for deliveries of human egocentric robotics video — episode validation, delivery diversity limits, and continuous streaming QA.

1. Introduction

1.1 What this system does

A *delivery* is a batch of egocentric video episodes captured by human operators wearing or holding an ARKit/iOS device. Before such a delivery can be paid for or trained on, two independent questions have to be answered, and they are not the same question:

Is each episode structurally sound? Does the metadata exist, parse, and agree with itself? Do the frame counts match across streams? Does the IMU actually cover the video? Is the episode identifiable? A delivery of ten thousand perfectly-varied episodes is worthless if the timestamps are broken.

Is the delivery as a whole varied enough? Ten thousand structurally-perfect episodes of one operator doing one task in one warehouse is also worthless. Diversity is a property of the *set*, not of any member of it, and it can only be measured in aggregate.

This system answers both, plus a third operational question — *is what is arriving right now sound?* — with a streaming watcher that validates each package as it lands.

1.2 Components

Component	Answers	Unit of judgement	Output
<code>schema_pipeline</code>	Is each episode structurally sound?	one capture session	ACCEPT / REVIEW / REJECT
<code>diversity_pipeline</code>	Is the delivery varied enough?	the whole delivery	PASS / FAIL per limit
<code>qa_watch</code>	Is what's arriving now sound?	one package, continuously	one MongoDB document
<code>reference_package</code>	What does a valid package look like?	40 files	passes every check

The two pipelines are genuinely independent. They share a design lineage and several module names, but neither imports the other, and either can be run without the other.

1.3 Dependencies

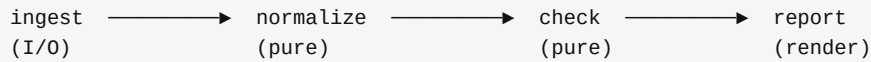
Both batch CLIs are **pure Python 3 standard library**. There is no `requirements.txt` for them and no `pip install` step. This is deliberate: the pipelines run in constrained environments — scheduled tasks, minimal containers, analyst laptops — where dependency installation is friction, and the S3 client is small enough to own outright (SigV4 signing is `hmac` plus `hashlib`; transport is `urllib`).

Requirement	Needed by	Optional?
Python 3.9+ (developed on 3.12)	everything	no
<code>ffprobe</code> on <code>PATH</code>	<code>diversity_pipeline</code>	yes — durations fall back to metadata timestamps, and each affected episode records that it did
<code>ffmpeg</code> on <code>PATH</code>	<code>reference_package</code> generator	yes — writes a placeholder video and says so
<code>pymongo</code> >= 4.6	<code>qa_watch</code>	no
<code>mongomock</code>	<code>qa_watch</code> tests	no

2. Architecture

2.1 Shape of both pipelines

Both pipelines are built the same way, and the shape is the point:



Exactly one module per pipeline touches the outside world. In `schema_pipeline` it is `pkg_ingest.py`; in `diversity_pipeline` it is `ingest.py` (plus `s3client` / `s3ingest` for the cloud path). Everything downstream — normalization, every check, every threshold comparison, aggregation — is pure over already-loaded data.

This is not architectural decoration. It means every check is testable without a filesystem, a network, or a fixture directory; a delivery can be validated from memory as easily as from disk (which is exactly what `qa_watch` does); and a check can never accidentally depend on I/O ordering, retry behaviour, or wall-clock time. Where a check *does* need the clock, it is injected — `validate.run(sessions, now_unix=...)` — so a time-dependent check is deterministic in tests.

2.2 Module map

`schema_pipeline/qa/` — 16 modules

Module	Lines	Responsibility
<code>pkg_ingest.py</code>	431	The only I/O module. Package discovery, directory <i>and</i> zip reading, fail-open JSON loading, chunk-envelope unwrapping, depth payload enumeration
<code>models.py</code>	148	<code>Package</code> (one chunk), <code>CaptureSession</code> (one episode)
<code>normalize.py</code>	112	Flatten to dotted paths; canonical key matching; priority-ordered resolution
<code>ruleset.py</code>	122	Every threshold and field location. Versioned separately from check logic
<code>sentinel.py</code>	88	Placeholder detection, fuzzy-matched by Levenshtein distance
<code>checks.py</code>	77	Check-record construction, severity → category mapping, verdict logic
<code>layerb.py</code>	437	Layer B: per-chunk metadata presence, validity, internal consistency
<code>layerc.py</code>	398	Layer C: cross-file referential and consistency checks
<code>session_checks.py</code>	176	Session-level: identity, chunk coherence, chunk continuity
<code>validate.py</code>	164	Orchestration, per-chunk finding collapse, delivery aggregation
<code>schema_report.py</code>	111	JSON / HTML / console rendering
<code>htmlkit.py</code>	98	Inline-CSS HTML helpers
<code>s3client.py</code> , <code>s3ingest.py</code>	380	Pure-stdlib SigV4 S3 client and cloud ingestion
<code>config.py</code> , <code>schema.py</code> , <code>profiles.py</code>	—	Shared with the diversity pipeline

`diversity_pipeline/qa/` — 11 modules

Module	Lines	Responsibility
<code>ingest.py</code>	238	The only local I/O module. Video/sidecar pairing, natural ordering, <code>ffprobe</code> duration measurement
<code>s3client.py</code>	213	Pure-stdlib SigV4 S3 client — list, get, HEAD, presign
<code>s3ingest.py</code>	167	Cloud-direct ingestion; two delivery layouts

Module	Lines	Responsibility
<code>metrics.py</code>	515	All nine diversity checks. Pure and I/O-free
<code>config.py</code>	136	Every threshold, alias and vocabulary
<code>profiles.py</code>	175	Per-delivery unwrap and confident aliases
<code>normalize.py</code>	95	Shared normalization
<code>models.py</code>	53	<code>Episode</code> dataclass
<code>schema.py</code>	137	Reference \$5 field registry
<code>report.py</code>	290	JSON / HTML / console rendering with charts
<code>htmlkit.py</code>	422	Charting primitives — donut, hbar, ratio bar, histogram, legend

2.3 The seven design principles

These are load-bearing. Several checks exist specifically to uphold them, and several *deliberate omissions* exist to avoid violating them.

1 — Never infer, only read. Declared fields are trusted and normalised for *spelling*; nothing is guessed from context.

`DIFFICULTY_SYNONYMS` maps `"med"` to `"medium"` because that is a spelling variation of a declared value. It does *not* map task names to difficulties, because that would be invention. An `environment_id` absent from the reference vocabulary is reported as unknown, never resolved to a plausible neighbour — the fix is to add the row to the table, not to guess.

2 — Absent is not passing. A check whose input is missing reports `NOT_COMPUTABLE` (diversity) or `BLOCKED` (schema), never `PASS`. This is why `business_size` limits report `NOT_COMPUTABLE` on a delivery that lacks the field rather than trivially passing: the distinction between *we checked and it was fine* and *we could not check* is the entire value of the report.

3 — Vendor fault is not the same as format gap. `FORMAT_GAP` marks a field the spec requires but the capture format has nowhere to put. That is a capture-app change request, not a data defect, and attributing it to the vendor would be wrong. It never blocks acceptance. If the field is present but wrong, that is an ordinary `ERROR`.

4 — Fail open on ingestion. A malformed side-car becomes a parse-error finding; the other files still load and the other checks still run. One corrupt `hands.json` must not cost you the verdict on the rest of the package.

5 — Measure; never trust a declaration. Rates and spans are computed from actual timestamp data. The clearest illustration is a deliberate *exclusion*: `motion_data.recording_duration` is left out of duration concordance because it is observed to be a constant `60.000` — the configured chunk length, not a measurement. Treating it as a declaration would fail every package in the delivery for a number nobody ever computed.

6 — Thresholds are data, versioned separately from logic. Every limit lives in `config.py` (diversity) or `ruleset.py` (schema). Tuning a threshold or moving a field's accepted location is a one-line edit that never touches check logic.

7 — Report checks that pass. Every check emits a record whatever the outcome, so *a check that did not run* stays distinguishable from *a check that passed*. A report with thirty `PASS` records is evidence; a report with silence is not.

3. Core concepts

3.1 Episode, session, chunk

This is the single most important structural fact about `schema_pipeline`, and getting it wrong was a real defect in an earlier revision.

An episode is a capture session. The capture app cuts a continuous recording into ~60-second **chunks**. Every side-car JSON is an array with one record per chunk:

```
[
  { "chunk": "chunk_0000_raw", "source": ".../chunk_0000_raw.zip", "data": { ... } },
  { "chunk": "chunk_0001_raw", "source": ".../chunk_0001_raw.zip", "data": { ... } },
  ...
]
```

A session's `metadata.json` therefore carries N metadata objects, its `video_timestamps.json` carries N timestamp arrays, and so on. One video file covers the whole session.

Validation runs at two levels:

- **Per chunk** — Layer B and Layer C, since every chunk has its own metadata, timestamps and intrinsics.
- **Per session** — identity, chunk coherence, chunk continuity, and the file checks that concern the session's single video and depth payloads.

Historical note. An earlier revision took only `[0].data` from each side-car, silently discarding every chunk after the first — around two thirds of a typical delivery. A second revision encoded a blocking decision "D-2: one episode == one chunk" and rejected any package with more than one metadata record, which rejected every well-formed session. Both are now fixed: N chunks is normal, and the real defect is *side-cars disagreeing on N* (check S2), because that means some stream is missing part of the recording.

Finding collapse

Per-chunk findings identical across chunks are merged into one record naming the affected chunks. Grouping is by `(id, severity, code, message)`, so genuinely distinct failures stay distinct — two chunks with different duration conflicts remain two records — while a defect repeated across every chunk becomes one:

```
ERROR B1.4 device_identified: device_type is missing [6 chunks: chunk_0000_raw,
chunk_0001_raw, chunk_0002_raw, chunk_0003_raw, +2 more]
```

Without this, a six-chunk session missing one field produces six identical lines, and a delivery report becomes unreadable noise.

Episode identity

A chunk's own `episode_uuid` field usually holds the *chunk label* (`"chunk_0498_raw"`), not a UUID. Session identity is therefore resolved from several candidates in precedence order — `manifest.doc_id`, a declared `session_id`, a declared `episode_uuid`, then the folder name — and **the first candidate that actually parses as a UUID wins**. Only if none parse does the first non-empty candidate become the identity, so a delivery that never writes a real UUID still gets a stable id and check B1.1 reports the truth about it rather than crashing or silently inventing one.

3.2 Canonical field resolution

You do not have to match spelling exactly. Before any check runs:

- 1. Flatten.** Nested objects become dotted paths: `data.metadata.task_id`. Only `dict` values are recursed into; lists and scalars are preserved as leaf values, so an array-valued field such as `camera_configuration` survives intact. An empty dict is itself a leaf, so its presence is not lost.
- 2. Canonicalise.** Keys are lowercased with `-`, `_`, `.` and whitespace stripped. So `task-id`, `task_id`, `Task_ID` and `taskId` all collapse to `taskid`.
- 3. Resolve,** in this precedence order — full dotted path, then the last (leaf) segment, then any interior segment. Only non-empty values count. An empty string, `null`, or an empty list/object counts as **absent**.
- 4. Alias priority.** Where a record carries two accepted spellings of the same field, the one listed **first in the registry** wins — never whichever key happens to appear first in the JSON. This matters more than it looks. `episode_uuid` accepts `session_id` as a *fallback*, deliberately last, because some capture apps carry the episode's UUID there. If resolution depended on JSON key order, a package declaring both would sometimes report the `episode_uuid` value and sometimes the `session_id` value, depending on how the app happened to serialise. With priority ordering, `episode_uuid` always wins — so an app writing the chunk name into that field is *reported by B1.1* rather than silently patched over from elsewhere.

3.3 Sentinel detection

Presence checks pass on placeholder values, which makes presence checking nearly useless on its own. Every string field validated for presence is additionally screened against a deny-list:

```
unknown · unknowncatagory · unknowncategory · na · none · null
tbd · todo · test · default · placeholder · undefined
```

Comparison strips all non-alphanumerics and lowercases, so `"N/A"`, `"n-a"` and `"NA"` are the same token.

Detection is deliberately **fuzzy**: terms of 5+ characters also match at Levenshtein distance ≤ 2 . The reason is concrete — a real reference sample carries `task_category: "Unknown_Catagory"`, which is both a sentinel *and* a misspelling, and an exact-match deny-list alone would wave it through. The 5-character floor stops short tokens like `na` and `tbd` from swallowing every 2–4 character legitimate value.

Two further rules: **a value equal to its own key name** is a sentinel (`"task_category": "task_category"`), and for `device_type` / `device_model` the generic names `iphone`, `android`, `phone` and `device` are additionally rejected, because they do not identify a hardware generation.

3.4 Severity model

Seven severities. The distinctions that matter are between *the vendor sent bad data*, *the capture format has nowhere to put this*, and *we could not evaluate this*.

Severity	Category	Meaning	Effect
ERROR	DATA_FAILURE	spec-required condition violated by delivered data	REJECT
BLOCKED	SPEC_GAP	cannot evaluate — missing prerequisite or spec gap	REVIEW
FORMAT_GAP	FORMAT_GAP	required field has no location in the capture format	none
CEILING	HARDWARE_CEILING	hardware-unmeetable requirement	none
WARN	WARNING	preferred condition unmet, recoverable ambiguity	none
INFO	INFO	observation, no judgement	none
PASS	PASS	the check ran and the condition held	none

Verdict rule: any `ERROR` → `REJECT`. Otherwise any `BLOCKED` → `REVIEW` (held for human review, *not* vendor fault). Otherwise `ACCEPT`, `WARN`, `INFO`, `CEILING` and `FORMAT_GAP` never block and are never counted as vendor data errors.

The diversity pipeline uses a separate, simpler status set — `PASS`, `FAIL`, `WARN`, `INFO`, `NOT_COMPUTABLE` — because its checks are aggregate limits rather than per-record conformance. Only `FAIL` sets a non-zero exit code.

4. schema_pipeline — episode validation

4.1 Invocation

```
cd schema_pipeline
python3 run.py <root> [--out OUTDIR] [--limit N] [--quiet]
```

Argument	Default	Meaning
<code>root</code>	<i>required</i>	a package directory, a chunk <code>.zip</code> , or a delivery directory whose immediate entries are either
<code>--out</code>	<code>qa_out</code>	output directory
<code>--limit N</code>	none	cap the number of packages validated (natural order)
<code>--quiet</code>	off	terser console output

Exit codes: `0` nothing rejected · `1` at least one REJECT · `2` usage / ingestion error.

4.2 Input layout

A folder is recognised as a package if it contains `metadata.json` or `manifest.json`, or if it holds a `chunk_*.zip`. Packages ship as directories **or zip archives**, and both are read without unpacking:

```
s3://<bucket>/<prefix>/
├─ a01524-H-0147/
│   ├── metadata.json      ← package_id
│   ├── manifest.json      ← required
│   ├── video.mov
│   ├── video_timestamps.json ← the reference clock
│   ├── camera_intrinsic.json
│   ├── motion_data.json
│   ├── depth_camera_intrinsics.json
│   ├── depth_00000.bin ... ← loose, or bundled in depth_maps.zip
│   ├── hands.json  hands_metadata.json
│   └─ ar_session.json
├─ EPISODE-F2094A73/
│   └─ chunk_0000_raw.zip ← package named after the EPISODE folder
└─ another-episode.zip    ← package named after the zip stem
```

An episode shipped as `<episode>/chunk_*.zip` is named after the **episode folder**, not the chunk — otherwise every episode in a delivery would be called `chunk_0000_raw`.

Archive members must sit at the archive root; nested directories are ignored, matching the flat-package rule used for directories.

`ar_session.json` is never fully materialised — only `len(frames)` is retained, and the payload is dropped immediately to bound memory on large deliveries.

4.3 Session-level checks

ID	Name	Rule	Severity on failure
S1	<code>session_metadata_present</code>	<code>metadata.json</code> exists with at least one chunk record	ERROR
S2	<code>chunk_coherence</code>	every side-car agrees on how many chunks there are, and on their names	ERROR
S3	<code>chunk_continuity</code>	chunk $k+1$ starts where chunk k ended, within 0.5 s	ERROR (BLOCKED if any chunk lacks a start time)

ID	Name	Rule	Severity on failure
B1.1	episode_uuid_valid	the resolved session identity parses as a UUID and is not a sentinel	ERROR
B1.1u	episode_uuid_unique	no episode_uuid is shared by two sessions in the delivery	ERROR (delivery-level pass)

On S2. Same count but different chunk *names* would mean streams describing different parts of the recording, so names are compared too wherever they are declared.

On S3. The tolerance is 0.5 s. One frame at 30 fps is 0.033 s; the observed capture app lands near 0.13 s, so 0.5 s absorbs real jitter without hiding a dropped chunk. Chunks are sorted by start time before comparison, and the report distinguishes a gap_s from an overlap_s.

On B1.1u. Uniqueness is enforced across the delivery *after* every session is validated. When a collision is found, the finding is appended to **every** session sharing the uuid, and each of their verdicts is recomputed — a duplicate is not one session's fault.

4.4 Layer B — metadata presence and validity

Per chunk. Every check returns exactly one record whatever the outcome.

ID	Field	Rule	On failure
B1.2	operator_id	non-empty, not a sentinel	ERROR
B1.3	episode_start_time	unix seconds or ISO-8601, within 1704067200 (2024-01-01) ... now + 24 h	ERROR
B1.4	device_type, device_model	both non-empty and not generic — bare "iPhone" fails	ERROR
B1.5	software_version	non-empty. Semver-like → PASS; present but not semver-like → WARN , not ERROR	ERROR / WARN
B1.6	camera_configuration	must name exactly the streams the package contains, in both directions	ERROR
B1.7	task_description	free-text description	FORMAT_GAP
B1.8	task_category	non-empty, not a sentinel, in the controlled vocabulary when one is configured	ERROR
B1.9	environment_label, environment_id	both non-empty and neither a placeholder — "Unknown" is rejected	ERROR
B1.10	indoor_outdoor	one of indoor, outdoor, mixed	FORMAT_GAP if absent; ERROR if present and unrecognised
B1.11	episode_end_time	strictly greater than episode_start_time	FORMAT_GAP if absent; ERROR if present and not after start
B2.1	task_id	non-empty	ERROR
B2.2	environment_id	non-empty	ERROR
B2.3	frame timestamps	non-empty and strictly monotonic	ERROR
B2.4	UTC anchor	agrees with episode_start_time within 1.0 s	ERROR
B2.5	latitude + longitude	both present (no range check today)	FORMAT_GAP
B2.6	subtask_annotations	non-empty	FORMAT_GAP
B2.7	instruction	non-empty, not a sentinel	FORMAT_GAP

B1.5 — why a WARN, not an ERROR

A `software_version` that exists but is not semver-shaped still identifies a build. The field's purpose is traceability, and an unusual format defeats *sorting*, not *traceability*. Missing entirely is an ERROR; oddly formatted is a WARN.

B1.6 — camera configuration matching

The pipeline infers which streams are present from the files themselves — `camera_intrinsic.json` present means `rgb`, `depth_camera_intrinsics.json` present means `depth` — and requires the declaration to match in **both** directions. A stream present but undeclared is an ERROR; a stream declared but absent is equally an ERROR.

Stream names are read from any of `stream`, `streams`, `name`, `type`, `cameras_present`, `camera_name` or `sensors`, recursing into nested lists. All of these work:

```
"camera_configuration": ["ego_camera", "depth"]
"camera_configuration": { "streams": ["rgb", "depth"] }
"camera_configuration": [ { "stream": "ego_camera" }, { "stream": "depth" } ]
"camera_configuration": [ { "mounting_locations": "head mounted",
                             "cameras_present": ["ego_camera", "lidar"] } ]
```

Tokens containing `rgb`, `ego`, `color`, `wide` or `camera` count as **rgb**. Tokens containing `depth`, `lidar` or `tof` count as **depth** — the capture app names the *sensor*, not the stream it produces, and treating `lidar` as an unknown token would fail every package that has a depth stream.

B2.4 — the UTC anchor

`episode_start_time` alone cannot be cross-checked. The anchor is looked for in `motion_data.json` under any of `recording_date`, `utc`, `utc_time`, `recorded_at`, `capture_time`, falling back to `manifest.json` → `generated_at`. It must agree with `episode_start_time` within 1.0 s.

4.5 Layer C — cross-file integrity

These are not in the source spec. They are derived from the observed package format and catch defects that per-field validation structurally cannot.

ID	Name	Rule	On failure
C1	<code>frame_count_concordance</code>	<code>video_timestamps</code> , <code>hands</code> , <code>ar_session</code> and <code>depth</code> frame counts must be equal . Fewer than 2 streams → BLOCKED, not PASS	ERROR
C2	<code>duration_concordance</code>	every declared <code>duration*</code> within 0.1 s of <code>ts[-1] - ts[0]</code>	ERROR
C3	<code>imu_covers_video_span</code>	IMU spans the video at both ends; max inter-sample gap $\leq 5 \times$ the median video frame interval	ERROR
C4	<code>referenced_files_exist</code>	every <code>depth_*.bin</code> named in the index exists	ERROR
C5	<code>single_authoritative_video</code>	exactly one video, or one declared authoritative	WARN (BLOCKED if none)
C6	<code>sensor_relation_declared</code>	with both RGB and depth present, declare the extrinsic transform	WARN
C7	<code>non_empty_payloads</code>	enabled hand streams must carry non-empty data	WARN
C8	<code>camera_intrinsics_declared</code>	intrinsics present when a video is, declaring all five geometry fields	ERROR
C9	<code>depth_geometry_declared</code>	every depth record declares <code>file</code> , <code>bytes_per_row</code> , <code>width</code> , <code>height</code>	ERROR

C1's scope note. The video's own `nb_frames` would require a decode — that is Layer A and out of scope here — so concordance is checked across the JSON-declared stream counts only.

C2's deliberate exclusion. `motion_data.recording_duration` is excluded, as described in §2.3 principle 5.

C3's columnar support. Some capture apps ship motion data as `{"columns": ["timestamp", "gyro_x", ...], "data": [[...], ...]}` rather than a named timestamps array. Both layouts are read, with the timestamp column located by name. Without this the samples would be invisible and C3 would report BLOCKED on a package that does carry IMU data.

C8 and C9 are presence-only, by decision. C8 confirms `focal_length`, `principal_point`, `image_width`, `image_height` and `intrinsic_matrix` are *declared*. It does not check whether `image_width` / `image_height` match the real video, nor whether `intrinsic_matrix` agrees with `focal_length` / `principal_point` — that needs the decoded video plus a numeric policy that has not been agreed. Likewise C9 confirms `bytes_per_row` is declared, never that `bytes_per_row × height` equals the payload's real byte size. These boundaries are deliberate and are restated in §10.

4.6 Worked example — a package that passes

```
[ { "chunk": "chunk_0000_raw", "data": { "metadata": {
  "episode_uuid":      "23DE884C-74DA-4BB1-A2E9-7E5B434CE7A3",
  "operator_id":       "OP-1837",
  "episode_start_time": 1784876817.882431,
  "episode_end_time":  1784876826.016731,
  "device_type":       "Apple iPhone 15 Pro",
  "device_model":      "iPhone15,2",
  "software_version":   "1.4.2",
  "camera_configuration": { "streams": ["ego_camera", "depth"] },
  "task_id":           "TASK-552",
  "task_category":     "Packaging",
  "task_description":  "Place mechanical parts into designated boxes",
  "instruction":        "Place mechanical parts into designated boxes",
  "environment_id":    "ENV-334",
  "environment_label": "warehouse",
  "indoor_outdoor":    "indoor",
  "latitude":          12.9716,
  "longitude":         77.5946,
  "subtask_annotations": [ { "t0": 0.0, "t1": 3.2, "label": "pick" } ],
  "duration":          8.1343,
  "sensor_relation":   { "rgb_to_depth": [1,0,0,0, 0,1,0,0, 0,0,1,0] }
} } } ]
```

With `video_timestamps` spanning 8.1343 s across 245 frames, `hands` / `ar_session` / `depth` also at 245 frames, IMU covering `[0, 8.30]`, and all 245 `depth*.bin` present, this returns **ACCEPT** with zero ERRORS.

Remove `episode_end_time`, `task_description`, `indoor_outdoor`, `latitude` / `longitude` and `subtask_annotations` and it **still ACCEPTs** — those five emit `FORMAT_GAP`, which never blocks.

5. diversity_pipeline — delivery diversity

5.1 Invocation

```
cd diversity_pipeline
python3 run.py <root> [--out OUTDIR] [--profile NAME] [--limit N]
                [--s3-layout folder|paired] [--factory-dynamic] [--quiet]
```

Argument	Default	Meaning
root	required	local directory, or s3://[bucket/]prefix for cloud-direct QA
--out	qa_out	output directory
--profile	generic	source profile — see §5.5
--limit N	none	cap episodes ingested (S3 only, natural order)
--s3-layout	folder	folder = per-episode <uid>/ folders; paired = videos/ + metadata/ matched by stem
--factory-dynamic	off	use the 2 h dynamic factory cap instead of the 1 h repetitive default
--quiet	off	terser console output

Exit codes: 0 all checks clear · 1 any FAIL · 2 usage / ingestion error.

5.2 Ingestion and pairing

Local. Videos (`.mp4` / `.mov` , case-insensitive) are paired with sidecars (`*.meta.json`) within a directory, keyed by (`dir` , `stem`) . Orphans — a video with no sidecar, or a sidecar with no video — are surfaced as episodes carrying a warning and are **never dropped**. An orphan is a finding, not an absence.

Episodes are ordered by the numeric part of the stem, so `video10` sorts after `video2` rather than before it.

Cloud-direct. No download. A single paginated listing groups keys by immediate folder — one request for the delivery rather than one per episode — then metadata is fetched concurrently (24 workers) and the video is referenced by a **presigned URL**, which `ffprobe` reads via HTTP range requests. A full delivery's durations are measured without transferring a single video.

5.3 Duration reconciliation

Every diversity limit is denominated in hours, so getting duration right is upstream of every check. Per episode:

Video measured?	Metadata span valid?	Hours used	Source	Warning
yes	yes	video	both	only if they disagree
yes	no	video	video	—
no	yes	metadata span	metadata	"no measured video duration"
no	no	0 h	none	"no duration available"

Where both exist, the measured video wins — it is the artefact, the timestamps are a claim about it. A disagreement is warned about only when it exceeds **1 second absolute and 2 % relative**; below either bound it is encoding jitter, not a defect.

Every episode's duration provenance appears in the report, so a delivery whose hours came mostly from metadata is visibly different from one whose hours were measured.

5.4 The nine checks

Check 1 — Environment category (L1)

Four limits on the distribution of hours across L1 environment categories:

Limit	Threshold
Any single category	≤ 20 % of hours
Top 5 categories combined	≤ 60 %
Top 10 categories combined	≤ 85 %
Distinct categories present	≥ 15

The report includes the full distribution and, separately, an *environment reference* table listing the 19-entry controlled vocabulary (Retail, Fashion, Repair, Food and beverage, Construction, Food processing, Factory, Hospitality, Automotive, Sports and recreation, Administrative, Healthcare, Cleaning, Laboratory, Agriculture, Beauty and personal care, Entertainment, Laundry, Warehouse) against what the delivery actually reached. Categories never captured are named explicitly — the distinct-count limit is a direct function of that gap — and any `environment_id` not in the vocabulary is listed last as unrecognised, never guessed.

Check 2 — Difficulty mix

Shares are of **known-difficulty hours**, not total hours; unknown-difficulty hours are excluded and reported separately.

Limit	Threshold
easy	≤ 30 %
hard	≥ 5 %
medium	40 % target — reported as INFO, not enforced

Any unknown-difficulty hours downgrade a passing result to `WARN`, so an apparently-good mix computed from a small labelled minority is visibly qualified.

Check 3 — Per-task share

Difficulty	Max share of total hours	Absolute cap
hard	6.0 %	200 h
medium	4.0 %	200 h
easy	1.5 %	200 h

Check 4 — Task × Environment

Difficulty	Cap
hard	40 h
medium	20 h
easy	10 h
factory pair	1 h (repetitive, default) or 2 h with <code>--factory-dynamic</code>

A pair is treated as factory when its `environment_l1` contains `factory`, `manufactur` or `industrial`. Factory work is repetitive by nature, so a flat cap replaces the difficulty-based one — an hour of the same assembly-line motion is an hour of the same assembly-line motion regardless of how the task was labelled.

Check 5 — Operator × Task × Environment

Difficulty	Cap
hard	10 h
medium	2 h
easy	1 h

Check 6 — Environment × Operator

Flat cap of **40 h** for any one operator in any one environment.

Check 7 — Business-size caps per environment

Requires `business_size` ; `NOT_COMPUTABLE` when the field is absent.

Size	Cap
small	500 h
medium	2 000 h
large	5 000 h

Check 8 — Workforce mix

Requires `worker_type` . By hours: real employees ≥ **70 %**, contractors ≤ **30 %**.

Check 9 — Human–human interaction

Requires the `human_interaction` flag. By **episode count** (not hours): ≥ **5 %** of episodes. This is a target, so falling short is a `WARN` , never a `FAIL` .

Mixed-difficulty groups

Where a group's episodes carry conflicting difficulty labels, the **most severe** label is used (`hard` > `medium` > `easy`) and the conflict count is reported in the check detail. Choosing the strictest interpretation means a mislabelled group is never let through by accident.

5.5 Source profiles

A profile adapts one delivery's metadata shape without touching the check logic. It declares two things: a pure `unwrap` transform from raw parsed JSON to the metadata object (which only reshapes structure — it never renames keys or invents values), and **confident field aliases** so a canonical field resolves to the delivery's own key.

Only confident mappings belong in a profile. Genuinely-absent fields are left to report honestly.

Profile	Delivery	Adaptation
<code>generic</code>	—	none (default)
<code>xphi-capture-v1</code>	Vultr <code>xphi-capture</code> <code>processed/</code>	<code>unwraps array</code> → <code>data.metadata</code> ; <code>episode_start_time</code> → <code>start_time_unix</code> , <code>environment_label</code> → <code>environment_l1</code>
<code>licensed-egocentric-v1</code>	Build.AI Egocentric-100K	split <code>videos/</code> + <code>metadata/</code> layout; <code>factory_id</code> → <code>environment_id</code> , <code>worker_id</code> → <code>operator_id</code>
<code>xphi-episode-v1</code>	xphi4 <code>egocentric_data/</code>	reads the rich <code>episode_metadata.json</code> sidecar rather than the sparse chunk- array <code>metadata.json</code> ; <code>environment</code> → <code>environment_l1</code>

xphi-capture-v1 deliberately does **not** map task_category, device_model, device_type, software_version or camera_configuration — those mappings would be guesses at semantically different fields. xphi-episode-v1 deliberately leaves business_size unmapped because it is genuinely absent, so that check stays NOT_COMPUTABLE rather than being fabricated into a pass.

5.6 Report contents

Beyond the nine check cards, the HTML report opens with a **delivery composition** section that describes the shape of the delivery *before* any pass/fail judgement — every figure derived from the same per-episode rows the checks use:

- Hours by environment and hours by task (donut, top 5 plus a folded "Other")
 - Difficulty mix as an ordinal ratio bar measured against the hard floor — ordered hard → easy, because the rule is a floor on hard
 - Workforce mix as a ratio bar against the real-employee floor
 - Operator concentration (horizontal bars, top 9 plus tail)
 - Episode length distribution (histogram, in minutes)
 - The environment reference table described in Check 1
 - A per-episode duration table with provenance and warnings
-

6. qa_watch — continuous validation

6.1 What it is for

Where the batch CLI answers *"was this delivery any good?"* after the fact, `qa_watch` answers *"is what is arriving right now any good?"* — one MongoDB document per package, queryable while capture is still in progress.

6.2 Guarantees

- **Source is read-only.** Only ever lists and gets from the source prefix. Never writes, never deletes.
- **Idempotent.** A package already in MongoDB is skipped, so restarts and overlapping polls never duplicate work or double-count findings. `_id` is the package id, which is what makes this true rather than merely intended.
- **Tiny disk footprint.** Per-package scratch, deleted immediately after validation.
- **One bad package cannot stop the loop.** Failures are recorded on the run document and the scan continues.
- **Clean shutdown.** `SIGTERM` / `SIGINT` finish the current package, then exit — safe for `docker stop` and rolling restarts.

6.3 Shadow packages

Layer B and Layer C never decode video and never read binary payloads. They need the JSON sidecars plus the *existence and size* of the binaries. So rather than pulling gigabytes per episode, `s3source.materialize` builds a **shadow package** in scratch:

Object	What happens
the nine JSON sidecars	downloaded for real (kilobytes)
<code>video.mp4</code> / <code>video.mov</code> / <code>depth_*.bin</code>	sparse file truncated to the object's true size — correct <code>st_size</code> , near-zero disk, zero bytes transferred

`pkg_ingest` then runs completely unmodified, and both the existence checks (C4, C5) and any future byte-size check see the truth. A typical episode costs a few hundred KB of transfer instead of several GB.

Caveat. This breaks the moment a check reads binary *content* — decoding video, or hashing a depth payload. Adding one of those means this module must start fetching for real.

6.4 Uniqueness across the stream

`episode_uuid` uniqueness cannot be judged from one package. The batch CLI checks it within a single delivery; here it is enforced against **everything ever seen**, by querying MongoDB. When a new package collides with an earlier one, the earlier documents are amended in place — they were correct on the evidence available at the time, and leaving them stale would be worse than amending them.

6.5 Data model

packages — one document per validated package

```
{
  _id: "a01524-H-0147",           // package id → idempotency key
  session_id: "a01524",
  episode_uuid: "0E0CC9E5-2928-4260-A7CD-0F2E6D7FB230",
  verdict: "REJECT",              // ACCEPT | REVIEW | REJECT
  counts: { ERROR: 6, BLOCKED: 4, FORMAT_GAP: 4, PASS: 13, ... },
  codes: ["DEVICE_UNDESPECIFIED", "SOFTWARE_VERSION_MISSING", ...],
  checks: [ { id, name, severity, category, code, observed, expected,
              source_file, spec_ref, message }, ... ],
  source_uri: "s3://xphi3/processed/a01524-H-0147/",
  source_stats: { objects, sidecars, binaries, bytes_fetched, bytes_referenced },
  validated_at: "2026-07-25T11:04:12Z",
  spec_version: "human-egocentric-umi-2026-04-09",
  pipeline_version: "0.1.0",
  stored_at: ISODate(...)
}
```

`codes` is the flattened list of non-`PASS` finding codes, and it is indexed — so “how many packages are missing a device type?” is one query rather than an `$unwind`.

runs — one document per watcher cycle

```
{ started_at, finished_at, status: "ok"|"error"|"interrupted",
  scanned, validated, skipped, failed, verdicts: {REJECT: 12}, errors: [...] }
```

A cycle that validated nothing *because the source was unreachable* looks different from one that simply had nothing new. That distinction is the entire point of this collection.

Indexes: `episode_uuid`, `verdict`, `session_id`, `validated_at` (desc), `codes`, and `runs.started_at` (desc).

6.6 Example queries

```
// current delivery health
db.packages.aggregate([ { $group: { _id: "$verdict", n: { $sum: 1 } } } ]])

// top findings, delivery-wide
db.packages.aggregate([
  { $unwind: "$codes" },
  { $group: { _id: "$codes", n: { $sum: 1 } } },
  { $sort: { n: -1 } }, { $limit: 15 }
])

// which sessions are worst
db.packages.aggregate([
  { $match: { verdict: "REJECT" } },
  { $group: { _id: "$session_id", rejected: { $sum: 1 } } },
  { $sort: { rejected: -1 } }
])

// is capture healthy right now?
db.runs.find().sort({ started_at: -1 }).limit(5)
```

7. reference_package

PKG-REFERENCE-0001 is a complete synthetic capture package — 40 files, 2.9 MB, 30 frames at 30 fps, span 0.9667 s — on which **every check passes**:

```
Packages: 1 accepted: 1 review: 0 rejected: 0
Findings: PASS=30
PKG-REFERENCE-0001 [ACCEPT]
```

Every consistency rule is satisfied by construction, which is the point — these are the relationships real capture software has to get right:

Property	Value	Check
Frame counts across all four streams	all 30	C1
Declared <code>duration</code> vs measured span	0.9667 s vs 0.9667 s	C2
IMU span vs video span	<code>[-0.01, 0.98]</code> covers <code>[0.0, 0.9667]</code> , max gap $0.01\text{ s} \leq 5 \times 0.0333$	C3
Depth <code>.bin</code> referenced vs present	30 vs 30	C4
Video files present	exactly one	C5
RGB + depth both present	<code>sensor_relation</code> declared	C6
Hand flags enabled	frames carry non-empty data	C7
<code>camera_configuration</code> vs streams	<code>["ego_camera", "depth"]</code> ↔ rgb + depth	B1.6
UTC anchor vs <code>episode_start_time</code>	identical	B2.4

The video is real H.264 at 1920×1440, encoded to the exact frame count and rate the timestamps declare, with intrinsics matching its real resolution and the principal point at the image centre. Nothing checks those today (see §10) — but a reference that contradicts itself would be a poor template.

The 30 depth payloads are byte-identical, so git stores a single blob for all of them: the repository cost is a few KB, not 2.9 MB.

7.1 Breaking it on purpose

Each edit starts from the valid package and fires one named check:

Edit	Result
<code>episode_uuid</code> → <code>"not-a-uuid"</code>	B1.1 ERROR → REJECT
<code>device_type</code> → <code>"iPhone"</code>	B1.4 ERROR — generic, doesn't identify hardware generation
<code>task_category</code> → <code>"Unknown_Catagory"</code>	B1.8 ERROR — sentinel caught despite the misspelling
<code>duration</code> → <code>60.0</code>	C2 ERROR — conflicts with the 0.9667 s span
delete one <code>depth_*.bin</code>	C4 ERROR — referenced file missing
delete <code>video_timestamps.json</code>	B2.3 ERROR, and C1/C2/C3 drop to BLOCKED
copy the folder under a second name	B1.1u ERROR on both — duplicate <code>episode_uuid</code>
remove <code>task_description</code>	B1.7 FORMAT_GAP — still ACCEPT, never blocks
add a second <code>video.mp4</code>	C5 WARN — ambiguous authoritative video

7.2 Regenerating

```
python3 reference_package/make_reference_package.py [dest_dir]
```

Constants at the top of the generator control the shape — `FRAME_COUNT` , `FPS` , `VIDEO_W/H` , `DEPTH_W/H` , `IMU_HZ` , `START_UNIX` . The frame count is small so the package stays browsable; a real capture runs to hundreds or thousands of frames and nothing else about the shape changes.

8. Output formats

8.1 schema_report.json

```
{
  "packages": [
    {
      "package_id": "a01524-H-0147",
      "episode_uuid": "0E0CC9E5-...",
      "episode_uuid_source": "manifest.json:doc_id",
      "chunk_count": 6,
      "chunk_names": ["chunk_0000_raw", ...],
      "validated_at": "2026-07-25T11:04:12Z",
      "pipeline_version": "0.1.0",
      "spec_version": "human-egocentric-umi-2026-04-09",
      "verdict": "REJECT",
      "counts": { "ERROR": 6, "WARN": 0, "BLOCKED": 4, "PASS": 13, ... },
      "checks": [
        { "id": "B1.4", "name": "device_identified", "severity": "ERROR",
          "category": "DATA_FAILURE", "code": "DEVICE_UNDESPECIFIED",
          "observed": "iPhone", "expected": "identifies hardware generation",
          "source_file": "metadata.json", "spec_ref": "Doc1 §5.3",
          "message": "device_type underspecified: generic device name ...",
          "chunks": ["chunk_0000_raw", ...], "chunk_count": 6 }
      ]
    }
  ],
  "aggregate": {
    "packages_total": 120, "chunks_total": 743,
    "verdicts": { "REJECT": 118, "ACCEPT": 2 },
    "packages_rejected": 118, "packages_review": 0, "packages_accepted": 2,
    "severity_totals": { "ERROR": 704, "FORMAT_GAP": 480, "PASS": 1560, ... },
    "top_findings": [ { "code": "DEVICE_UNDESPECIFIED", "count": 118 }, ... ]
  }
}
```

`spec_ref` is emitted on **every** record, pass or fail, so any finding can be traced to the clause it came from.

8.2 diversity_report.json

```
{
  "checks": [
    { "name": "Environment category (L1)", "status": "FAIL",
      "detail": "distinct=3, top-5=100.0%, top-10=100.0%; ...",
      "offenders": [ { "label": "warehouse", "hours": 210.4, "pct": 62.1 } ],
      "extra": { "distribution": [ ... ] } }
  ],
  "summary": {
    "episodes": 1204, "total_hours": 338.72,
    "status_counts": { "PASS": 2, "FAIL": 4, "NOT_COMPUTABLE": 3 },
    "any_fail": true, "factory_dynamic": false
  },
  "rows": [
    { "stem": "...", "hours": 0.2813, "duration_source": "video",
      "task_id": "...", "operator_id": "...", "environment_id": "...",
      "environment_l1": "...", "difficulty": "medium", "business_size": null,
      "worker_type": "real", "human_interaction": false, "warnings": [] }
  ]
}
```

`rows` is the complete per-episode basis for every aggregate, so any figure in the report can be recomputed or audited independently.

8.3 HTML

Both pipelines write a single self-contained HTML file with inline CSS — no external assets, no network fetches, no build step. It can be emailed, attached to a ticket, or opened from a file:// URL years later and still render. The diversity report additionally renders charts as inline SVG, each paired with a collapsible table view so the underlying numbers are always one click away.

9. Configuration

Credentials are read from the environment and are **never** hardcoded or committed.

Variable	Used by	Default	Purpose
<code>AWS_ACCESS_KEY_ID</code>	diversity, qa_watch	<i>required for S3</i>	access key
<code>AWS_SECRET_ACCESS_KEY</code>	diversity, qa_watch	<i>required for S3</i>	secret key
<code>S3_ENDPOINT</code>	diversity, qa_watch	<i>required for S3</i>	host only, e.g. <code>blr2.vultrobjects.com</code>
<code>AWS_REGION</code>	diversity, qa_watch	<code>us-east-1</code>	e.g. <code>blr2</code>
<code>AWS_S3_BUCKET_NAME</code>	diversity, qa_watch	—	default bucket when a path omits it
<code>MONGO_URI</code>	qa_watch	<i>required</i>	connection string
<code>MONGO_DB</code>	qa_watch	<code>qa</code>	database name
<code>SRC</code>	qa_watch	<i>required</i>	source prefix, e.g. <code>s3://xphi3/processed</code>
<code>WORK</code>	qa_watch	<code>/tmp/qawork</code>	scratch directory
<code>POLL_SECONDS</code>	qa_watch	<code>300</code>	seconds between scans
<code>ONE_SHOT</code>	qa_watch	<code>0</code>	<code>1</code> = drain backlog once, then exit
<code>BATCH_LIMIT</code>	qa_watch	<code>0</code>	max packages per cycle, <code>0</code> = no cap
<code>SESSION_RE</code>	qa_watch	<code>^([\^~]+)-</code>	regex whose group 1 is the session id
<code>SCHEMA_PIPELINE_DIR</code>	qa_watch	auto	override the path to <code>schema_pipeline</code>

9.1 Tuning thresholds

Every limit is data, not logic.

To change	Edit
a diversity limit	<code>diversity_pipeline/qa/config.py</code>
the environment vocabulary	<code>ENVIRONMENT_NAMES</code> in the same file
a difficulty / worker-type / business-size spelling	the corresponding <code>*_SYNONYMS</code> dict
a schema threshold or tolerance	<code>schema_pipeline/qa/ruleset.py</code>
where a field is allowed to live	<code>_FIELD_ALIASES</code> in <code>ruleset.py</code> — order matters , first wins
the placeholder deny-list	<code>_SENTINELS</code> in <code>sentinel.py</code>
an accepted side-car filename	<code>SIDECAR_NAMES</code> in <code>pkg_ingest.py</code>

`SPEC_VERSION` and `PIPELINE_VERSION` in `ruleset.py` are emitted on every report, so a report always carries the ruleset revision that produced it.

10. Scope boundaries

Things the pipelines read but deliberately do **not** validate. This section exists because silence must not be mistaken for correctness.

Not checked	Why
Intrinsic values	C8 confirms declaration only. Comparing <code>image_width / image_height</code> against the real video, or <code>intrinsic_matrix</code> against <code>focal_length / principal_point</code> , needs a decode (Layer A) and a numeric policy that has not been agreed
Depth stride	C9 confirms <code>bytes_per_row</code> is declared, never that <code>bytes_per_row × height</code> equals the payload's real size. A 0-byte <code>.bin</code> still passes C4
IMU sample rate	only gaps relative to the video frame interval are checked
Frame-rate stability	B2.3 only requires timestamps to increase, so a clip dropping 60 fps → 5 fps mid-recording passes
<code>latitude / longitude</code> range	presence only
<code>episode_end_time - episode_start_time</code> vs measured span	only ordering is enforced
Video content	never decoded — that is Layer A, out of scope throughout

A known soundness gap. Four checks — B1.6, C4, C6, C7 — return `PASS` rather than `BLOCKED` when their inputs are absent. An empty package therefore scores passes it did not earn. This is a genuine defect rather than a decision, and is the highest-value correctness fix available in the check layer.

11. Extending the system

11.1 Adding a Layer B check

1. Write `def bX_Y_name(pkg: Package, ctx: Context) -> dict` in `layerb.py`. Return exactly one record via `check(...)` or `passed(...)`, whatever the outcome.
2. Put any threshold it needs in `ruleset.py`, never inline.
3. Append it to `CHECKS` at the bottom of the module.

The function is pure over an already-loaded `Package`. If you find yourself wanting to open a file, the work belongs in `pkg_ingest` instead.

11.2 Adding a diversity check

1. Write `def check_name(rows: List[DivRow]) -> Dict[str, Any]` in `metrics.py`, returning `_check(name, status, detail, offenders, extra)`.
2. Return `NOT_COMPUTABLE` when the input field is absent — never `PASS`.
3. Put thresholds in `config.py`; add the check to the list in `run()`.

If the check needs a field not yet on `DivRow`, add it to the dataclass, to `DIVERSITY_ALIASES`, and to `build_row`.

11.3 Adding a source profile

Add a `SourceProfile` to `profiles.py` with an `unwrap` (structure only — never renaming keys or inventing values) and *confident* aliases only, then register it in `_REGISTRY`. It becomes available as a `--profile` choice automatically.

The discipline that makes profiles safe is refusing to map uncertain fields. A profile that force-maps a semantically-different field to satisfy a check converts an honest `NOT_COMPUTABLE` into a false `PASS`, which is strictly worse than no profile at all.

11.4 Adding an accepted spelling

One line in `_FIELD_ALIASES` (`ruleset.py`) or `DIVERSITY_ALIASES` (`config.py`). Remember that alias order is priority order: the canonical name goes first, fallbacks last.

12. Testing

```
cd schema_pipeline
python3 tests/test_reference_fixture.py      # per-check verdicts on the reference fixture
python3 tests/test_capture_app_formats.py    # formats the iOS capture app actually emits

pip install pymongo mongomock
python3 qa_watch/test_qa_watch.py           # fake S3 + mongomock, no live services
python3 reference_package/test_reference_package.py
```

All fixtures are synthetic and built at runtime. No capture data is committed and no live services are required.

`test_capture_app_formats.py` is worth reading on its own: each case corresponds to a real difference found between an actual capture package and the reference contract, grouped into structure (chunk zips, bundled depth archives, plural intrinsic filename), naming (`start_timestamp` / `total_duration` / `session_id` carrying data the validator was looking for under other names) and vocabulary (`cameras_present: ["lidar"]` , placeholder environment labels).

12.1 Current test status

Verified on this tree. **Both batch CLIs work correctly** — the reference package validates to `ACCEPT` with 30 PASS and exit 0. The failures below are confined to `qa_watch` and the test suites, and all share one root cause: the validator was refactored from *one episode = one package* to *one episode = one session of N chunks*, and these callers were not migrated with it.

Location	Symptom	Impact
<code>qa_watch/qa_watch.py:96-99</code>	calls <code>pkg_ingest.load_package()</code> and passes the resulting <code>Package</code> to <code>validate.validate_package()</code> , which now requires a <code>CaptureSession</code> → <code>AttributeError: 'Package' object has no attribute 'chunk_count'</code>	the watcher crashes on every package ; 10 of 12 tests pass, the 2 exercising validation fail
<code>tests/test_reference_fixture.py</code>	calls <code>pkg_ingest.discover_packages()</code> , renamed to <code>discover_sessions()</code>	4 of 4 tests error; expectations also cite check <code>D2</code> , replaced by <code>S2</code>
<code>tests/test_capture_app_formats.py</code>	calls <code>validate.validate_package()</code> with a <code>Package</code>	24 of 28 tests error
<code>reference_package/test_reference_package.py</code>	same call pattern	errors before running
<code>INPUT_REFERENCE.md</code> §2, §7	still documents <code>D2</code> ("one episode must be one chunk")	stale — deliberately replaced by <code>S2/S3</code>

The fix in each case is to move to `load_session()` / `validate_session()` and re-derive the handful of expectations that referenced the retired `D2` rule.

Appendix A — Check index

Schema pipeline

ID	Name	Level
S1	session_metadata_present	session
S2	chunk_coherence	session
S3	chunk_continuity	session
B1.1	episode_uuid_valid	session
B1.1u	episode_uuid_unique	delivery
B1.2	operator_id_valid	chunk
B1.3	episode_start_time_plausible	chunk
B1.4	device_identified	chunk
B1.5	software_version_present	chunk
B1.6	camera_config_complete	chunk
B1.7	task_description_present	chunk
B1.8	task_category_valid	chunk
B1.9	environment_labelled	chunk
B1.10	indoor_outdoor_declared	chunk
B1.11	episode_end_time_present	chunk
B2.1	task_id_present	chunk
B2.2	environment_id_present	chunk
B2.3	frame_timestamps_present	chunk
B2.4	utc_reference_present	chunk
B2.5	geo_location_present	chunk
B2.6	subtask_annotations_present	chunk
B2.7	instruction_present	chunk
C1	frame_count_concordance	chunk
C2	duration_concordance	chunk
C3	imu_covers_video_span	chunk
C4	referenced_files_exist	session
C5	single_authoritative_video	session
C6	sensor_relation_declared	chunk
C7	non_empty_payloads	chunk
C8	camera_intrinsics_declared	chunk
C9	depth_geometry_declared	chunk

Diversity pipeline

#	Name	Requires
1	Environment category (L1)	environment_l1
2	Difficulty mix	task_difficulty
3	Per-task share	task_id
4	Task × Environment	task_id, environment_id
5	Operator × Task × Environment	operator_id, task_id, environment_id
6	Environment × Operator	environment_id, operator_id
7	Business-size caps	business_size, environment_id
8	Workforce mix	worker_type
9	Human–human interaction	human_interaction

Appendix B — Finding codes

PARSE_ERROR · METADATA_MISSING · CHUNK_COUNT_MISMATCH · CHUNK_DISCONTINUITY ·
EPISODE_UUID_INVALID · EPISODE_UUID_DUPLICATE · OPERATOR_ID_INVALID · START_TIME_IMPLAUSIBLE ·
DEVICE_UNDESPECIFIED · SOFTWARE_VERSION_MISSING · CAMERA_CONFIG_MISMATCH ·
TASK_DESCRIPTION_MISSING · TASK_CATEGORY_INVALID · ENVIRONMENT_LABEL_MISSING ·
INDOOR_OUTDOOR_MISSING · END_TIME_MISSING · TASK_ID_MISSING · ENVIRONMENT_ID_MISSING ·
FRAME_TIMESTAMPS_INVALID · UTC_REFERENCE_INCONSISTENT · GEO_LOCATION_MISSING ·
SUBTASK_ANNOTATIONS_MISSING · INSTRUCTION_MISSING · FRAME_COUNT_MISMATCH ·
DURATION_DECLARATION_CONFLICT · IMU_COVERAGE_GAP · REFERENCED_FILE_MISSING ·
AMBIGUOUS_VIDEO_SOURCE · SENSOR_RELATION_UNDECLARED · EMPTY_PAYLOAD · CAMERA_INTRINSICS_MISSING ·
DEPTH_GEOMETRY_UNDECLARED